

tcheyna

Guide d'installation et de déploiement

Lancer la plateforme en local,
puis la mettre en production

La location immobilière, en toute confiance
Conakry, République de Guinée

Version 1.1 · 07/09/2026
Jean Faya Kamano, fondateur

Sommaire

- 1 Ce que contient le projet**
L'architecture en deux applications
- 2 Prérequis**
Ce qu'il faut installer avant de commencer
- 3 Lancer l'application en local**
Backend, frontend et comptes de démonstration
- 4 Mettre en production**
Base de données Supabase et hébergement Render
- 5 Activer les services tiers**
SMS Africa's Talking et paiements CinetPay
- 6 Vérifier que tout fonctionne**
Contrôles après installation ou déploiement
- 7 Résoudre les problèmes courants**
Symptômes, causes et corrections
- 8 Aide-mémoire des commandes**
Les commandes utiles au quotidien

À qui s'adresse ce document

Il décrit deux parcours : faire tourner Tcheyna sur un poste de développement (sections 1 à 3), et le déployer sur Internet (sections 4 et 5). Les deux se suivent dans l'ordre. Aucune connaissance préalable du projet n'est supposée.

SECTION 1

Ce que contient le projet

Tcheyna est constitué de **deux applications indépendantes** qui communiquent par une API REST. Elles vivent dans le même dépôt mais se lancent et se déploient séparément.

ARBORESCENCE

```
Tcheyna/
|
+-- backend/           API REST - Python / Flask
|   +-- app/
|       | +-- models.py    Utilisateurs, annonces, passeports...
|       | +-- routes/     Un fichier par domaine métier
|       | +-- services/   Moteur de matching, notifications
|   +-- config.py
|   +-- run.py          Point d'entrée du serveur
|   +-- seed.py         Données de démonstration
|   +-- requirements.txt
|
+-- frontend/          Interface web - React / Vite
|   +-- src/
|       | +-- api/         Appels au backend
|       | +-- components/  Éléments réutilisables
|       | +-- pages/       Écrans locataire, propriétaire, admin
|   +-- package.json
|
+-- render.yaml        Configuration de déploiement
+-- README.md
```

Rôle de chaque application

Application	Rôle	Port en local
backend	Contient toute la logique métier : niveaux de confiance, certification des annonces, matching, paiements. C'est la seule à parler à la base de données.	5000
frontend	Affiche l'interface et appelle le backend. Ne contient aucune règle métier : il ne fait qu'afficher ce que l'API renvoie.	5173

Pourquoi cette séparation

Les règles de confiance et les montants sont calculés côté serveur uniquement. Un utilisateur ne peut donc pas contourner un niveau de vérification ni modifier le prix qu'il paie en manipulant la page web.

SECTION 2

Prérequis

Trois outils à installer avant toute chose. Les versions indiquées sont les minimums testés.

Outil	Version	À quoi il sert	Où l'obtenir
Python	3.11+	Faire tourner le backend	python.org/downloads
Node.js	20+	Faire tourner le frontend	nodejs.org
Git	—	Récupérer et versionner le code	git-scm.com

Vérifier que tout est bien installé

```
python --version    # doit afficher 3.11 ou plus
node --version      # doit afficher v20 ou plus
git --version
```

Sous Windows

À l'installation de Python, cochez impérativement la case « **Add Python to PATH** ». Sans elle, la commande python reste introuvable dans le terminal.

Aucun compte externe n'est nécessaire pour démarrer. Les services de SMS et de paiement (section 5) sont facultatifs : sans eux, l'application fonctionne entièrement, en mode dégradé documenté.

SECTION 3

Lancer l'application en local

Le lancement se fait en **deux terminaux séparés**, un par application. Laissez les deux ouverts pendant toute la session de travail.

Étape 1 — Récupérer le code

TERMINAL

```
git clone <adresse-du-depot> Tcheyna
cd Tcheyna
```

Étape 2 — Démarrer le backend (terminal n° 1)

TERMINAL 1 – BACKEND

```
cd backend

# Créer un environnement Python isolé
python -m venv .venv

# L'activer sous Windows :
.venv\Scripts\activate
# L'activer sous Linux ou macOS :
source .venv/bin/activate

# Installer les dépendances
pip install -r requirements.txt

# Créer le fichier de configuration
copy .env.example .env          # Windows
cp .env.example .env           # Linux / macOS

# Créer la base et les données de démonstration
python seed.py

# Démarrer le serveur
python run.py
```

Le backend est prêt quand...

le terminal affiche Running on `http://0.0.0.0:5000`. Vérifiez en ouvrant `http://localhost:5000/health` dans un navigateur : la page doit afficher `{"status": "ok"}`.

Étape 3 — Démarrer le frontend (terminal n° 2)

TERMINAL 2 – FRONTEND

```
cd frontend
npm install
npm run dev
```

L'application est accessible

sur **http://localhost:5173**. Aucune configuration n'est nécessaire côté frontend en local : les appels vers `/api` sont automatiquement relayés vers le backend.

Comptes de démonstration

La commande `python seed.py` a créé six comptes couvrant tous les rôles et tous les niveaux de confiance. Le mot de passe est le même pour tous : **tcheyna2026**

Adresse e-mail	Rôle	Particularité
admin@tcheyna.test	Administrateur	Accès au back-office de vérification
aminata@tcheyna.test	Locataire	Niveau 3 — dossier complet et solvable
mamadou.t@tcheyna.test	Locataire	Niveau 2 — identité vérifiée
fatoumata@tcheyna.test	Locataire	Niveau 1 — téléphone vérifié
ibrahima@tcheyna.test	Propriétaire	3 annonces dont 2 certifiées
mariama@tcheyna.test	Propriétaire	2 annonces, 1 en attente de certification

Parcours conseillé pour une démonstration

Connectez-vous en **ibrahima** pour voir les candidatures reçues et accepter un dossier — la messagerie s'ouvre alors automatiquement. Puis passez en **aminata** pour découvrir les annonces recommandées avec leur score de compatibilité. Terminez en **admin** pour certifier une annonce en attente.

Réinitialiser les données

Relancer `python seed.py` efface les données de démonstration et les recrée à l'identique. Les comptes créés manuellement pendant vos essais ne sont pas conservés.

SECTION 4

Mettre en production

La mise en ligne repose sur deux services gratuits : **Supabase** pour la base de données PostgreSQL, et **Render** pour héberger les deux applications. Comptez une trentaine de minutes la première fois.

Étape 1 — Créer la base de données sur Supabase

- Créez un compte sur **supabase.com**, puis un nouveau projet.
- Choisissez un mot de passe de base de données et **conservez-le** : il n'est plus affiché ensuite.
- Ouvrez *Settings* → *Database* → *Connection string* → *URI* et copiez l'adresse obtenue. Elle commence par `postgresql://`

Étape 2 — Déployer sur Render

- Poussez le projet sur GitHub s'il ne l'est pas déjà.
- Sur **render.com**, choisissez *New* → *Blueprint* et sélectionnez le dépôt.
- Render lit le fichier `render.yaml` à la racine et crée automatiquement les deux services : l'API et le site web.

Étape 3 — Renseigner les variables secrètes

Dans le tableau de bord du service **tcheyna-api**, onglet *Environment*, complétez les valeurs suivantes. Les autres variables sont déjà définies par `render.yaml`.

Variable	Valeur à saisir	Obligatoire
DATABASE_URL	L'adresse copiée depuis Supabase à l'étape 1	Oui
CORS_ORIGINS	L'adresse publique du frontend, par exemple <code>https://tcheyna.onrender.com</code>	Oui
AT_API_KEY	Clé Africa's Talking — voir section 5	Non
CINETPAY_API_KEY	Clé CinetPay — voir section 5	Non
CINETPAY_SITE_ID	Identifiant de site CinetPay	Non

Point de vigilance

CORS_ORIGINS doit correspondre exactement à l'adresse du frontend, sans barre oblique finale. Une erreur ici est la cause la plus fréquente d'une interface qui s'affiche mais reste vide : le navigateur bloque alors tous les appels à l'API.

Étape 4 — Créer les tables et le compte administrateur

Une fois l'API déployée, ouvrez le *Shell* du service **tcheyna-api** depuis le tableau de bord Render, puis exécutez :

```
SHELL RENDER — TCHEYNA-API
# Créer les tables dans la base Supabase
flask init-db

# Créer votre compte administrateur
python create_admin.py
```

Le script demande une adresse e-mail, un nom et un mot de passe. Ce compte donne accès au back-office : validation des pièces d'identité, des justificatifs de revenus et certification des annonces.

Ne pas exécuter `seed.py` en production

Ce script **efface les données existantes** avant de créer les comptes de démonstration. Il est réservé au développement local.

Le stockage des documents

Le fichier `render.yaml` déclare un disque persistant monté sur `/var/data`. Les pièces d'identité et justificatifs téléversés par les utilisateurs y sont conservés et survivent aux redéploiements. Sans ce disque, ces documents seraient perdus à chaque mise à jour du code.

Mettre à jour l'application

Render surveille la branche `main`. Chaque `git push` déclenche automatiquement un nouveau déploiement des deux services. Aucune action manuelle n'est nécessaire.

SECTION 5

Activer les services tiers

Deux intégrations donnent au produit ses fonctions clés : la vérification par SMS et le paiement Mobile Money. Elles sont facultatives — l'application fonctionne sans elles, avec le comportement de repli décrit ci-dessous.

Service	Ce qu'il apporte	Comportement sans la clé
Africa's Talking africastalking.com	Envoi du code de vérification par SMS, qui fait passer un utilisateur au niveau 1 du Passeport Locataire.	Le code est renvoyé directement dans la réponse de l'API en mode développement. Le parcours reste testable de bout en bout.
CinetPay cinetpay.com	Paiement par Orange Money et MTN MoMo pour la caution, le premier loyer et les services Tcheyna.	Le paiement est enregistré au statut « en attente » au lieu d'échouer. Le reste du parcours se poursuit normalement.

Obtenir les clés

- **Africa's Talking** — créez un compte, puis relevez la clé d'API dans *Settings* → *API Key*. Le mode « sandbox » permet de tester sans frais avant de passer en production.
- **CinetPay** — créez un compte marchand ; la clé d'API et l'identifiant de site figurent dans le tableau de bord, section *Intégration*.

Renseigner les clés

En local, dans le fichier backend/ .env. **En production**, dans l'onglet *Environment* du service tcheyna-api sur Render.

```
BACKEND/.ENV
AT_API_KEY=votre_cle_africas_talking
AT_USERNAME=sandbox
AT_SENDER_ID=TCHEYNA

CINETPAY_API_KEY=votre_cle_cinetpay
CINETPAY_SITE_ID=votre_identifiant_de_site
```

Sécurité des paiements

Les montants sont **toujours calculés par le serveur** : le client ne choisit jamais combien il paie. De même, les notifications de paiement envoyées par CinetPay ne sont pas prises pour argent comptant — le statut réel est revérifié auprès de CinetPay avant d'être appliqué.

SECTION 6

Vérifier que tout fonctionne

Après une installation ou un déploiement, ces quatre contrôles confirment que l'ensemble de la chaîne est opérationnel.

	Contrôle	Résultat attendu
1	Ouvrir /health sur l'API	La page affiche {"status": "ok"}
2	Ouvrir la page d'accueil du frontend	La page se charge, sans zone vide ni message d'erreur
3	Se connecter avec un compte	L'arrivée sur le tableau de bord se fait sans erreur
4	Consulter la liste des annonces	Les annonces s'affichent avec leur prix en GNF

Contrôles techniques

```
# Le code frontend est-il cohérent ?
cd frontend && npm run typecheck

# Le frontend se construit-il pour la production ?
cd frontend && npm run build

# L'API répond-elle ?
curl http://localhost:5000/health
```

Une remarque sur Render

L'offre gratuite met le service en veille après quinze minutes sans trafic. La première visite qui suit peut demander **trente à cinquante secondes** le temps du réveil : c'est normal, ce n'est pas une panne. Un service de surveillance comme UptimeRobot, pointé sur /health, permet de limiter ces mises en veille.

SECTION 7

Résoudre les problèmes courants

Symptôme	Cause probable	Correction
L'interface s'affiche mais reste vide, sans annonces	Le backend n'est pas démarré, ou CORS_ORIGINS ne correspond pas à l'adresse du frontend	Vérifier que le backend tourne ; corriger CORS_ORIGINS pour qu'il reproduise exactement l'adresse du frontend, sans barre oblique finale
ModuleNotFoundError au lancement du backend	L'environnement Python n'est pas activé	Réactiver .venv, puis relancer <code>pip install -r requirements.txt</code>
Le code SMS n'arrive pas	Aucune clé Africa's Talking n'est configurée	Comportement normal : en développement, le code est renvoyé directement à l'écran par l'API
Le paiement reste « en attente »	Aucune clé CinetPay n'est configurée	Comportement normal en développement. En production, vérifier la clé et l'adresse de notification
Le port 5000 est déjà utilisé	Une autre application occupe ce port	Fermer l'application concernée, ou définir un autre port via la variable PORT
Rechargement d'une page interne : erreur 404 en production	Le service statique ne redirige pas vers index.html	Vérifier que la règle de réécriture est bien présente dans <code>render.yaml</code>

SECTION 8

Aide-mémoire des commandes

Backend

```
cd backend
.venv\Scripts\activate           # Windows
source .venv/bin/activate        # Linux / macOS

pip install -r requirements.txt   # Installer les dépendances
python seed.py                   # Réinitialiser les données de démo
python create_admin.py           # Créer un administrateur
python run.py                    # Démarrer le serveur
flask init-db                    # Créer les tables sans données
```

Frontend

```
cd frontend
npm install                      # Installer les dépendances
npm run dev                      # Démarrer en développement
npm run typecheck                # Vérifier le code
npm run build                    # Construire pour la production
npm run preview                  # Prévisualiser la version construite
```

Adresses utiles

Environnement	Frontend	API
Local	localhost:5173	localhost:5000
Production	tcheyna.onrender.com	tcheyna-api.onrender.com

Pour aller plus loin

Le fichier README.md, à la racine du projet, détaille l'architecture technique : système de confiance à quatre niveaux, fonctionnement du moteur de matching, liste complète des points d'entrée de l'API et modèle économique.

Tcheyna — La location immobilière, en toute confiance

Conakry, République de Guinée · Jean Faya Kamano, fondateur · jeafatkamano123@gmail.com